

LAPORAN PENGUJIAN
TEOREMA CAP PADA SISTEM TERDISTRIBUSI
Studi Perbandingan Consistency, Availability, dan Partition Tolerance pada
MongoDB dan Apache Cassandra



Disusun oleh:
NAMA : JUSRIADI LIWANG
NIM : 105841117023
KLS RPL 6-A

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH MAKASSAR 2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan pengujian ini dapat disusun dengan baik. Laporan ini membahas pengujian konsep Teorema CAP pada sistem terdistribusi, khususnya dalam membandingkan perilaku sistem yang mengutamakan Consistency dan Partition Tolerance serta sistem yang mengutamakan Availability dan Partition Tolerance. Pengujian ini dilakukan untuk memahami bagaimana sistem database terdistribusi memberikan respons ketika terjadi gangguan komunikasi jaringan antar node. Pada laporan ini, MongoDB digunakan sebagai contoh sistem yang cenderung menunjukkan karakteristik CP, sedangkan Apache Cassandra digunakan sebagai contoh sistem yang menunjukkan karakteristik AP.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar laporan ini dapat menjadi lebih baik.

Makassar 21 Juni 2026

Jusriadi liwang

DAFTAR ISI

BAB I.....	4
PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Rumusan Masalah	4
1.3 Tujuan Pengujian	5
1.4 Manfaat Pengujian.....	5
BAB II	6
LANDASAN TEORI.....	6
2.1 Sistem Terdistribusi.....	6
2.2 Teorema CAP	6
2.3 Model CP	7
2.4 Model AP	7
2.5 MongoDB dan Apache Cassandra.....	7
BAB III	8
METODE PENGUJIAN	8
3.1 Objek Pengujian	8
3.2 Alat dan Bahan	8
3.3 Skenario Pengujian.....	8
3.4 Indikator Pengujian	9
BAB IV	10
HASIL DAN PEMBAHASAN	10
4.1 Hasil Pengujian MongoDB sebagai Model CP.....	10
4.2 Analisis Pengujian MongoDB	11
4.3 Hasil Pengujian Apache Cassandra sebagai Model AP	11
4.4 Analisis Pengujian Apache Cassandra.....	12
4.5 Tabel Perbandingan Hasil Pengujian.....	13
4.6 Pembahasan Umum.....	13
BAB V	14
PENUTUP	14
5.1 Kesimpulan	14
5.2 Saran	14
DAFTAR PUSTAKA	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi digital membuat kebutuhan terhadap sistem yang cepat, stabil, dan mampu melayani banyak pengguna semakin meningkat. Aplikasi modern seperti marketplace, media sosial, layanan pembayaran, sistem kesehatan, dan layanan cloud umumnya tidak lagi berjalan hanya pada satu server, melainkan menggunakan sistem terdistribusi. Sistem terdistribusi memungkinkan beban kerja dibagi ke beberapa server sehingga layanan dapat berjalan lebih cepat dan skalabel. Namun, dalam sistem terdistribusi terdapat tantangan penting, yaitu bagaimana sistem tetap berjalan ketika terjadi gangguan jaringan antar server. Gangguan tersebut dapat menyebabkan sebagian node tidak dapat saling berkomunikasi, sehingga sistem harus menentukan prioritas antara menjaga data tetap konsisten atau tetap memberikan layanan kepada pengguna.

Teorema CAP menjelaskan bahwa dalam sistem terdistribusi, ketika terjadi gangguan jaringan atau partition, sistem tidak dapat menjamin Consistency, Availability, dan Partition Tolerance secara bersamaan. Oleh karena itu, sistem harus memilih apakah akan lebih mengutamakan konsistensi data atau ketersediaan layanan. Berdasarkan hal tersebut, laporan ini dibuat untuk menguji dan menganalisis perilaku dua pendekatan sistem, yaitu CP dan AP. MongoDB digunakan untuk melihat perilaku sistem yang lebih mengutamakan konsistensi data, sedangkan Apache Cassandra digunakan untuk melihat perilaku sistem yang tetap menjaga ketersediaan layanan meskipun terjadi gangguan jaringan.

1.2 Rumusan Masalah

Rumusan masalah dalam laporan ini adalah sebagai berikut:

- a. Bagaimana konsep Teorema CAP diterapkan pada sistem terdistribusi?
- b. Bagaimana perilaku MongoDB ketika terjadi gangguan jaringan antar node?

- c. Bagaimana perilaku Apache Cassandra ketika terjadi gangguan jaringan antar node?
- d. Apa perbedaan utama antara pendekatan CP dan AP dalam menghadapi partition?

1.3 Tujuan Pengujian

Tujuan dari pengujian ini adalah:

- a. Memahami konsep Consistency, Availability, dan Partition Tolerance dalam Teorema CAP.
- b. Menguji respons MongoDB ketika terjadi gangguan jaringan.
- c. Menguji respons Apache Cassandra ketika terjadi gangguan jaringan.
- d. Membandingkan kelebihan dan kelemahan pendekatan CP dan AP.
- e. Menentukan dampak pemilihan model CAP terhadap aplikasi nyata.

1.4 Manfaat Pengujian

Manfaat dari pengujian ini adalah memberikan pemahaman praktis mengenai perilaku sistem database terdistribusi. Dengan memahami Teorema CAP, pengembang dapat menentukan jenis database dan arsitektur sistem yang sesuai dengan kebutuhan aplikasi, baik yang lebih mengutamakan keakuratan data maupun yang lebih mengutamakan ketersediaan layanan.

BAB II

LANDASAN TEORI

2.1 Sistem Terdistribusi

Sistem terdistribusi adalah sistem yang terdiri atas beberapa komputer atau server yang saling terhubung melalui jaringan dan bekerja sama untuk menyediakan layanan kepada pengguna. Sistem ini banyak digunakan karena mampu meningkatkan skalabilitas, ketersediaan, dan keandalan layanan.

Dalam sistem terdistribusi, data dapat disimpan pada beberapa node. Hal ini memberikan keuntungan karena sistem tidak bergantung pada satu server saja. Namun, sistem juga harus mampu mengatasi masalah ketika terjadi gangguan jaringan antar node.

2.2 Teorema CAP

Teorema CAP diperkenalkan oleh Eric Brewer pada tahun 2000. Teorema ini menyatakan bahwa sistem terdistribusi tidak dapat secara bersamaan menjamin tiga aspek utama, yaitu Consistency, Availability, dan Partition Tolerance, terutama ketika terjadi gangguan jaringan. Tiga komponen utama dalam CAP adalah:

a. Consistency

Consistency atau konsistensi berarti semua node dalam sistem memiliki data yang sama dan terbaru. Ketika data diperbarui pada satu node, maka node lain juga harus memiliki data yang sama agar tidak terjadi perbedaan informasi. Sistem yang mengutamakan konsistensi cocok digunakan pada aplikasi yang membutuhkan keakuratan data tinggi, seperti sistem perbankan, payment gateway, dan data medis.

b. Availability

Availability atau ketersediaan berarti sistem selalu memberikan respons kepada pengguna, meskipun ada sebagian node yang gagal atau tidak dapat berkomunikasi. Sistem yang mengutamakan availability akan tetap melayani permintaan pengguna walaupun data yang diberikan mungkin belum sepenuhnya sinkron. Sistem seperti ini cocok digunakan pada aplikasi media sosial, marketplace, sistem rekomendasi, dan mesin pencari.

c. Partition Tolerance

Partition Tolerance berarti sistem tetap dapat berjalan meskipun terjadi gangguan komunikasi antar node. Dalam sistem terdistribusi, partition menjadi aspek penting karena gangguan jaringan dapat terjadi kapan saja, terutama pada sistem berbasis cloud.

2.3 Model CP

Model CP merupakan pendekatan yang mengutamakan Consistency dan Partition Tolerance. Ketika terjadi gangguan jaringan, sistem CP lebih memilih menolak sebagian request daripada memberikan data yang tidak konsisten.

Kelebihan model CP adalah data lebih akurat dan risiko inkonsistensi dapat dikurangi. Namun, kekurangannya adalah layanan dapat mengalami downtime sementara ketika sistem tidak dapat menjamin sinkronisasi data antar node.

2.4 Model AP

Model AP merupakan pendekatan yang mengutamakan Availability dan Partition Tolerance. Ketika terjadi gangguan jaringan, sistem AP tetap memberikan respons kepada pengguna meskipun data yang diberikan mungkin belum sinkron sepenuhnya.

Kelebihan model AP adalah sistem tetap online dan downtime sangat kecil. Namun, kekurangannya adalah terdapat risiko pembacaan data lama atau stale data sampai proses sinkronisasi selesai.

2.5 MongoDB dan Apache Cassandra

MongoDB merupakan database NoSQL yang dapat digunakan dalam sistem terdistribusi. Pada kondisi tertentu, MongoDB dapat menunjukkan karakteristik CP karena sistem akan menolak operasi tertentu jika node tidak berstatus primary atau tidak dapat menjamin konsistensi data. Apache Cassandra merupakan database NoSQL terdistribusi yang dirancang untuk memiliki availability tinggi. Cassandra lebih cocok untuk sistem yang membutuhkan layanan tetap aktif meskipun terjadi gangguan pada sebagian node.

BAB III

METODE PENGUJIAN

3.1 Objek Pengujian

Objek pengujian dalam laporan ini adalah dua sistem database terdistribusi, yaitu:

- MongoDB sebagai contoh sistem dengan karakteristik CP.
- Apache Cassandra sebagai contoh sistem dengan karakteristik AP.

3.2 Alat dan Bahan

Alat dan bahan yang digunakan dalam pengujian ini adalah:

- Laptop atau komputer.
- Docker.
- Docker Compose.
- MongoDB.
- Apache Cassandra.
- Terminal atau Command Prompt.
- Jaringan Docker untuk simulasi gangguan komunikasi antar node.

3.3 Skenario Pengujian

Pengujian dilakukan dengan mensimulasikan gangguan jaringan atau partition. Simulasi dilakukan dengan memutus koneksi salah satu node dari jaringan menggunakan perintah Docker. Setelah itu, dilakukan pengujian terhadap respons sistem.

Skenario pengujian dibagi menjadi dua bagian:

a. Pengujian CP pada MongoDB

Pada pengujian ini, sistem MongoDB dijalankan dengan beberapa node. Dalam kondisi normal, data yang disimpan pada node utama akan direplikasi ke node lain. Setelah itu, dilakukan pemutusan koneksi pada salah satu node untuk mensimulasikan kondisi partition. Pengujian dilakukan dengan mencoba melakukan operasi penulisan data baru. Respons sistem diamati untuk melihat apakah MongoDB tetap menerima request atau menolak request demi menjaga konsistensi data.

b. Pengujian AP pada Apache Cassandra

Pada pengujian ini, sistem Apache Cassandra dijalankan dalam bentuk node terdistribusi. Setelah sistem berjalan, dilakukan simulasi partition dengan memutus salah satu node dari jaringan. Kemudian dilakukan pembacaan data menggunakan perintah query. Respons sistem diamati untuk melihat apakah Cassandra tetap memberikan layanan meskipun salah satu node berada dalam kondisi terisolasi.

3.4 Indikator Pengujian

Indikator pengujian yang digunakan adalah:

Respons sistem saat terjadi partition.

Apakah sistem menerima atau menolak request.

Prioritas sistem terhadap konsistensi atau ketersediaan.

Risiko teknis yang muncul dari masing-masing pendekatan.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Pengujian MongoDB sebagai Model CP

Pada kondisi normal, MongoDB mampu menyimpan data pada node utama dan mereplikasi data tersebut ke node lain. Hal ini menunjukkan bahwa data antar node dapat tersinkronisasi ketika jaringan berjalan dengan baik. Namun, ketika dilakukan simulasi partition dengan memutus koneksi pada salah satu node, MongoDB menunjukkan perilaku yang berbeda. Saat dilakukan operasi penulisan data baru, sistem memberikan pesan kesalahan seperti `MongoServerError: not primary`. Hasil ini menunjukkan bahwa MongoDB tidak menerima operasi penulisan ketika node tidak dapat menjamin status primary dan sinkronisasi data. Dengan kata lain, MongoDB lebih memilih menolak request sementara daripada menerima data yang berpotensi menyebabkan perbedaan antar node. Berdasarkan hasil tersebut, MongoDB menunjukkan karakteristik CP. Sistem lebih mengutamakan konsistensi data dan toleransi terhadap partition, meskipun harus mengorbankan availability untuk sementara waktu.

```
PS D:\Data Pindahan C:\Desktop\Desktop\CAP-Pengujian-Kelompok4> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
4e518f2b6206   cassandra:4.1 "docker-entrypoint.s..." 4 days ago    Up 23 seconds 0.0.0.0:9043->9042/tcp, [::]:9043->9042/tcp  cap-cassandra-node2
9eb31441ffc5   cassandra:4.1 "docker-entrypoint.s..." 4 days ago    Up 23 seconds 0.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp  cap-cassandra-node1
b4071c5fc5bf   mongo:7        "docker-entrypoint.s..." 4 days ago    Up 23 seconds 27017/tcp                           cap-mongo-node1
b435bf0b414    mongo:7        "docker-entrypoint.s..." 4 days ago    Up 23 seconds 0.0.0.0:27018->27017/tcp, [::]:27018->27017/tcp  cap-mongo-node2
45d10ac37689   mongo:7        "docker-entrypoint.s..." 4 days ago    Up 23 seconds 0.0.0.0:27019->27017/tcp, [::]:27019->27017/tcp  cap-mongo-node3
```

Gambar 4.1 Tampilan Container MongoDB dan Cassandra Berjalan

Gambar ini menunjukkan bahwa container MongoDB dan Apache Cassandra berhasil berjalan menggunakan Docker sebagai lingkungan pengujian sistem terdistribusi.

```
PS D:\Data Pindahan C:\Desktop\Desktop\CAP-Pengujian-Kelompok4> docker exec cap-mongo-node1 mongosh --quiet --eval "db
mal MongoDB sebelum partition", waktu:new Date()); printjson(db.users.find().toArray())"
MongoServerError: not primary
PS D:\Data Pindahan C:\Desktop\Desktop\CAP-Pengujian-Kelompok4>
```

```
PS D:\Data Pindahan C\Desktop\Desktop\CAP-Pengujian-Kelompok4> .\scripts\06-pulihkan-mongodb.ps1
=== MEMULIHKAN MONGODB ===
Menyambungkan kembali cap-mongo-node1 ke cap-network...
Menunggu sinkronisasi ulang...
Error response from daemon: container b1829c8b6a2dd17b139ba5d03e784e23707c9214cbffcc71073908d8fe6ae682 is not running

MongoDB sudah dipulihkan.
PS D:\Data Pindahan C\Desktop\Desktop\CAP-Pengujian-Kelompok4> █
```

Gambar 4.2 Pengujian MongoDB Setelah Partition

Gambar ini menunjukkan bahwa MongoDB menolak operasi penulisan ketika node mengalami gangguan jaringan. Error not primary menunjukkan bahwa MongoDB lebih mengutamakan konsistensi data dibandingkan ketersediaan layanan sementara.

4.2 Analisis Pengujian MongoDB

Perilaku MongoDB dalam pengujian ini menunjukkan bahwa sistem yang mengutamakan CP sangat cocok digunakan pada aplikasi yang membutuhkan keakuratan data. Contohnya adalah sistem perbankan, sistem pembayaran, dan sistem inventory medis. Keuntungan dari pendekatan CP adalah data yang tersimpan lebih aman dan tidak mudah mengalami inkonsistensi. Namun, kekurangannya adalah sistem dapat menolak request ketika terjadi gangguan jaringan. Hal ini dapat menyebabkan layanan tidak tersedia sementara bagi pengguna. Dengan demikian, pendekatan CP lebih cocok digunakan pada aplikasi yang tidak boleh memberikan data salah, meskipun layanan harus berhenti sementara.

4.3 Hasil Pengujian Apache Cassandra sebagai Model AP

Pada pengujian Apache Cassandra, dilakukan simulasi partition dengan memutus koneksi salah satu node dari jaringan utama. Setelah itu, dilakukan operasi pembacaan data menggunakan query. Berbeda dengan MongoDB, Cassandra tetap memberikan respons terhadap permintaan data meskipun node sedang berada dalam kondisi terisolasi. Query pembacaan data tetap berhasil dijalankan dan sistem masih dapat memberikan hasil kepada pengguna. Hasil ini menunjukkan bahwa Cassandra lebih mengutamakan ketersediaan layanan. Walaupun terdapat risiko data yang dibaca belum sinkron dengan node lain, sistem tetap memberikan respons kepada pengguna. Berdasarkan hasil tersebut,

Apache Cassandra menunjukkan karakteristik AP, yaitu mengutamakan Availability dan Partition Tolerance.

```
PS D:\Data Pindahan C\Desktop\Desktop\CAP-Pengujian-Kelompok4> .\scripts\08-uji-cassandra-normal.ps1
=== UJI CASSANDRA KONDISI NORMAL ===

id | keterangan | nama
-----+-----+-----
1 | Data normal Cassandra sebelum partition | Jusri

(1 rows)

Jika data tampil, Cassandra berjalan normal.
Lanjut ke 09-simulasi-partisi-cassandra.ps1
```

Gambar 4.3 Pengujian Cassandra pada Kondisi Normal

Gambar ini menunjukkan bahwa Apache Cassandra berhasil menampilkan data pada kondisi jaringan normal. Data dengan nama Jusri dapat dibaca dari tabel `cap_demo.users`, sehingga Cassandra berjalan dengan baik sebelum dilakukan simulasi partition

4.4 Analisis Pengujian Apache Cassandra

Perilaku Cassandra menunjukkan bahwa sistem AP cocok digunakan pada aplikasi yang membutuhkan layanan tetap aktif dan tidak boleh sering mengalami downtime. Contohnya adalah media sosial, sistem rekomendasi, marketplace, dan search engine. Keuntungan dari pendekatan AP adalah sistem tetap online meskipun terjadi gangguan pada sebagian node. Namun, kekurangannya adalah data yang diberikan kepada pengguna dapat berupa data lama atau stale data sampai proses sinkronisasi selesai. Dengan demikian, pendekatan AP lebih cocok untuk aplikasi yang lebih mengutamakan layanan tetap berjalan daripada keakuratan data secara langsung pada saat itu juga.

```
PS D:\Data Pindahan C\Desktop\Desktop\CAP-Pengujian-Kelompok4> .\scripts\10-uji-cassandra-setelah-partisi.ps1
=== UJI CASSANDRA SETELAH PARTITION ===
Menguji query lokal system.local pada node yang terisolasi...

key | bootstrapped | listen_address | data_center | rack
-----+-----+-----+-----+-----
local | COMPLETED | 172.21.0.2 | datacenter1 | rack1

(1 rows)

Menguji data aplikasi pada node yang terisolasi...

id | keterangan | nama
-----+-----+-----
1 | Data normal Cassandra sebelum partition | Jusri

(1 rows)

HASIL YANG DIHARAPKAN:
- Cassandra tetap memberikan respons data lokal.
- Ini menunjukkan karakteristik AP: layanan tetap tersedia walaupun ada risiko data belum sinkron.
```

Gambar 4.4 Pengujian Cassandra Setelah Partition

Gambar ini menunjukkan bahwa Apache Cassandra tetap memberikan respons data lokal meskipun node sedang berada dalam kondisi terisolasi dari jaringan. Data dengan nama Jusri masih dapat ditampilkan setelah partition dilakukan. Hal ini menunjukkan karakteristik AP, yaitu sistem tetap menjaga ketersediaan layanan walaupun terdapat risiko data belum sinkron sepenuhnya

4.5 Tabel Perbandingan Hasil Pengujian

Aspek Pengujian	MongoDB	Apache Cassandra
Model CAP	CP	AP
Fokus Utama	Konsistensi data	Ketersediaan layanan
Respons saat partition	Menolak operasi tertentu	Tetap memberikan respons
Contoh respons	Error not primary	Query berhasil dijalankan
Kelebihan	Data lebih akurat	Sistem tetap online
Kekurangan	Layanan dapat berhenti sementara	Risiko stale data
Cocok untuk	Bank, payment gateway, data medis	Media sosial, marketplace, search engine

4.6 Pembahasan Umum

Berdasarkan hasil pengujian, dapat dipahami bahwa tidak ada sistem terdistribusi yang dapat menjamin Consistency, Availability, dan Partition Tolerance secara sempurna pada saat terjadi gangguan jaringan. Sistem harus memilih prioritas sesuai kebutuhan aplikasi. MongoDB dalam pengujian ini menunjukkan bahwa sistem CP lebih berhati-hati terhadap perubahan data. Sistem menolak request ketika tidak dapat memastikan konsistensi data. Hal ini penting untuk aplikasi yang sangat bergantung pada data akurat. Sementara itu, Apache Cassandra menunjukkan bahwa sistem AP lebih mengutamakan agar layanan tetap berjalan. Sistem tetap memberikan respons walaupun terdapat kemungkinan data belum sepenuhnya sinkron. Hal ini penting untuk aplikasi berskala besar yang membutuhkan uptime tinggi. Pemilihan antara CP dan AP harus disesuaikan dengan kebutuhan bisnis. Jika aplikasi membutuhkan data yang sangat akurat, maka pendekatan CP lebih tepat. Namun, jika aplikasi lebih membutuhkan layanan selalu tersedia, maka pendekatan AP lebih sesuai

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan pengujian yang telah dilakukan, dapat disimpulkan bahwa Teorema CAP merupakan konsep penting dalam perancangan sistem terdistribusi. Ketika terjadi gangguan jaringan, sistem tidak dapat menjamin Consistency, Availability, dan Partition Tolerance secara bersamaan. MongoDB menunjukkan karakteristik CP karena sistem lebih mengutamakan konsistensi data dan menolak operasi tertentu ketika tidak dapat menjamin sinkronisasi antar node. Hal ini membuat MongoDB cocok untuk aplikasi yang membutuhkan akurasi data tinggi. Apache Cassandra menunjukkan karakteristik AP karena sistem tetap memberikan respons kepada pengguna meskipun terjadi gangguan jaringan. Hal ini membuat Cassandra cocok untuk aplikasi yang membutuhkan ketersediaan layanan tinggi. Dengan demikian, pemilihan database dan arsitektur sistem harus disesuaikan dengan kebutuhan aplikasi. Aplikasi yang membutuhkan data akurat lebih cocok menggunakan pendekatan CP, sedangkan aplikasi yang membutuhkan layanan tetap aktif lebih cocok menggunakan pendekatan AP.

5.2 Saran

Saran untuk pengujian selanjutnya adalah melakukan simulasi dengan jumlah node yang lebih banyak agar hasil pengujian lebih mendekati kondisi sistem nyata. Selain itu, pengujian dapat dikembangkan dengan menambahkan pengukuran waktu respons, jumlah request yang berhasil, jumlah request yang gagal, dan waktu pemulihan sistem setelah jaringan kembali normal. Pengujian juga dapat diperluas dengan membandingkan database lain seperti HBase, CouchDB, DynamoDB, PostgreSQL, atau MySQL agar pemahaman tentang penerapan Teorema CAP pada berbagai jenis database menjadi lebih lengkap.

DAFTAR PUSTAKA

Brewer, E. A. 2000. Toward Robust Distributed Systems. Proceedings of the Annual ACM Symposium on Principles of Distributed Computing.

Gilbert, S., & Lynch, N. 2002. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News.

Materi Presentasi Kelompok 4. Teorema CAP: Consistency, Availability, dan Partition Tolerance pada Scalable Systems Design.